ME 305 Mechatronics

Department of Mechanical Engineering

Cal Poly

Lab 4 Report: Function Generator

Written by Gabe Choi and Maddie Howard

11/6/25

Overview/Objective:

The objective of lab 4 is to use the 9S12XD timer module to design a function generator using time channel 0, interrupts and the digital to analogue converter (DAC) on our kits. Our program needs to accept a digit from the keypad for the user to indicate which waveform to display. 1 corresponded to a Sawtooth wave, 2 corresponded to a sine wave with 7 segments, 3 corresponded to a square wave and 4 corresponded to a sine wave with 15 segments. The program needs to then prompt the user to input a one-byte integer (NINT) using the keypad, which is the number of interrupts per Basic Time Interval (BTI). All our waveforms used a data table stored in flash memory (ROM) with our code. The table tables for each of the 4 waveforms are described in the report below. The Sawtooth and Square waveforms need to display at 500 Hz for NINT=1 and vary from a minimum of 0 volts to a maximum of 8 volts.

Much of the code needed for this lab was taken from our lab 3, including much of Task #1 MasterMind, Task #2 Keypad Driver and Task #3 Display, with the necessary variables changed to complete the objectives of this lab. Most of the new code included in our program was written in Task #4 Timer Channel 0 and Task #5 Function Generator. As with lab 3, our code needed to handle the ENTER key and BACKSPACE key, be user friendly and incorporate cooperative multitasking between each of the tasks.

Tasks:

Task #1: MasterMind

Mastermind is the brain of the whole code. It is in charge to direct and control all the other tasks to ensure that the overall objective of the program is running efficiently.

- State 0 – Initialization State. This includes all the hardware and software initialization for Mastermind. State 0 also clears all the display and input flags and requests the first menu display prompt.
- State 1 – Waiting State. In state 1, the program will wait until all the initializations in state 0 are completed and the first display prompt has been completed by the display task.
- State 2 – Hub State. This state checks to see whether a struck key is waiting to be dealt with. If the key has been struck, the ASCII value is determined, and the next state of MasterMind is selected. If no keys have been struck, then the program returns to *main,* and the next task is executed. The keys struck can either be for which waveform has been selected or for the NINT entry, so the code handles each case as needed
- State 3 – NINT Entry. Once the waveform has been selected and the program is ready for the NINT entry, the hub will send the program to this state. State 3 handles the digits for NINT, backspace and the enter keys. This state will also check to see if BUF is full and stores the digit in BUF if there is room. State 3 will also set waveform pointed based on the waveform selected in the Hub and raise the flags for the errors messages if needed.

Task #2: Keypad Driver

The keypad driver monitors the keypad for any struck keys from the user and reports the ASCII value of each of the keys back to MasterMind.

- State 0 – Initialization State. This state clears the key flag raised by MasterMind and initializes the keypad hardware.
- State 1 – Key Press State. In state 1, the program checks to see if the LKEY_FLG has been pressed, meaning a key has been detected. If a key has been pressed, then it reads the ASCII value and adds it into the key buffer flag, which is a signal to MasterMind to process the digit.
- State 2 – Waiting State. Task 2 will stay in state 2 after detecting a key, until MasterMind has cleared the key flag and therefore acknowledge that task 2 can return to state 1 and wait for the next key.

Task #3: Display

The display task carries out any printing of messages or values on the LCD display. In the spirit of cooperative multitasking, it is important that the display task only prints one character on the display per run. This meant that our code needed to implement "handshakes" between tasks, so other tasks could run concurrently while the display was being updated.

- State 0 – Initialization State. This state initializes the LCD hardware.
- State 1 – Hub State. The Hub checks all the display flags and routes to which handler is needed for the display messages. It also runs through to ensure that if an error message occurs that the ERR_ACTIVE flag is false before running the display prompts again on line 2.
- State 2 – Display Handler State. State 2 runs whatever prompt the Hub State requested. If it is an error message, the handler-state also makes sure that the error message only stays on the display for 1 second before going away.

Task #4: Timer Channel 0 Setup

Task 4 configures timer channel 0 for the output compare interrupts every 0.1ms. This task only runs once since it only needs to be initialized. This task only has one state, and it completes all the need steps to enable the timer channel and interrupts for the function generator.

Task #5: Function Generator

The function generator manages all the waveform generation logic and updates the waveform segments each BTI. Each waveform has a corresponding data table located in ROM and explained below in the report.

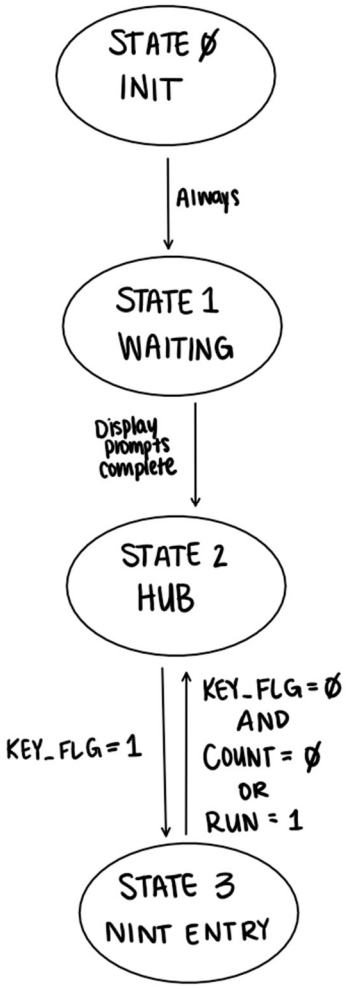
- State 0 – Initialization State. This state clears RUN and NEWBIT and then moves the task to state 1.
- State 1 – Wait for Wave Selection State. This state waits for the wave flag to be true before moving the task to state 2.
- State 2 – Load Waveform State. Once the wave flag has been raised, state 2 loads the parameters from the specified waveform table in ROM and loads all of the necessary variables for the waveform generation. It then asks Display to show the NINT prompt and moves the task to state 3.
- State 3 – Wait for NINT Entry State. State 3 waits for the display to be done and the NINT value to be validated. Then it allows the task to move to state 4.
- State 4 – Waveform Generation State. Once everything has been approved by the other tasks and states, the waveform is able to be generated by state 4. This state runs through each segment cycle to generate the waveform.

Inter-task Communication Variables:

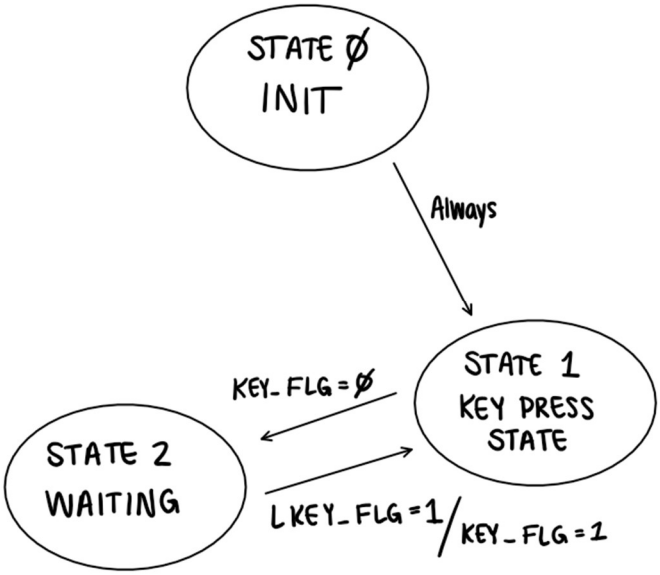
Variable Name	Variable Description	Task and State Set	Task and State Cleared/Changed
DWAVE	Flag for initial prompt message on line 1	Task 1 State 0	Task 3 State 2
WAVE_FLG	Flag for the selected waveform. 1 = Saw, 2 = Sine-7, 3 = Square, 4 = Sine-15	Task 1 State 1	Task 1 State 2
DPRMPT	Flag for display to show the prompt for the waveform on line 2	Task 3 State 1	Task 3 State 2
DSAW	Flag for Saw prompt	Task 1 State 2	Task 3 State 2
DSINE7	Flag for Sine-7 prompt	Task 1 State 2	Task 3 State 2
DSQUARE	Flag for Square prompt	Task 1 State 2	Task 3 State 2
DSINE15	Flag for Sine-15 prompt	Task 1 State 2	Task 3 State 2
KEY_FLG	Flag showing a key is available	Task 2 State 1	Task 1 State 2
KEY_BUF	Holds the ASCII value of the last key pressed	Task 2 State 1	Task 1 State 2
DIGIT_FLG	Flag to echo a digit	Task 1 State 3	Task 3 State 2
BSPACE_FLG	Flag that backspace has been pressed	Task 1 State 3	Task 3 State 2
DLARGEMSG	Flag for Magnitude too Large error message prompt	Task 1 State 3	Task 3 State 2
DZEROMSG	Flag for Zero Magnitude error message prompt	Task 1 State 3	Task 3 State 2
DNOINPUT	Flag for No Digits Entered error message prompt	Task 1 State 3	Task 3 State 2
ERR_ACTIVE	Flag for if the error message is active or not	Task 3 State 1	Task 3 State 2
RUN	Function generator enabled	Task 1 State 3	Task 5 State 0
NEWBIT	Flag for if a new BTI is inputted	Interrupt	Task 5 State 4
CNT	Number of interrupts remaining in this BTI	Task 5 State 3	Interrupt
CSEG	Remaining number of segments	Task 5 State 2	Task 5 State 4
LSEG	Remaining number of BTI's in this segment	Task 5 State 2	Task 5 State 4
SEGINC	Input to DAC	Task 5 State 2	Task 5 State 4
SEGPTR	Address of the first line of the next segment in the wave data for this wave	Task 5 State 2	Task 5 State 4
WAVEPTR	Address of the first line of wave data for this wave	Task 1 State 3	Task 5 State 4

Finite State Machines:

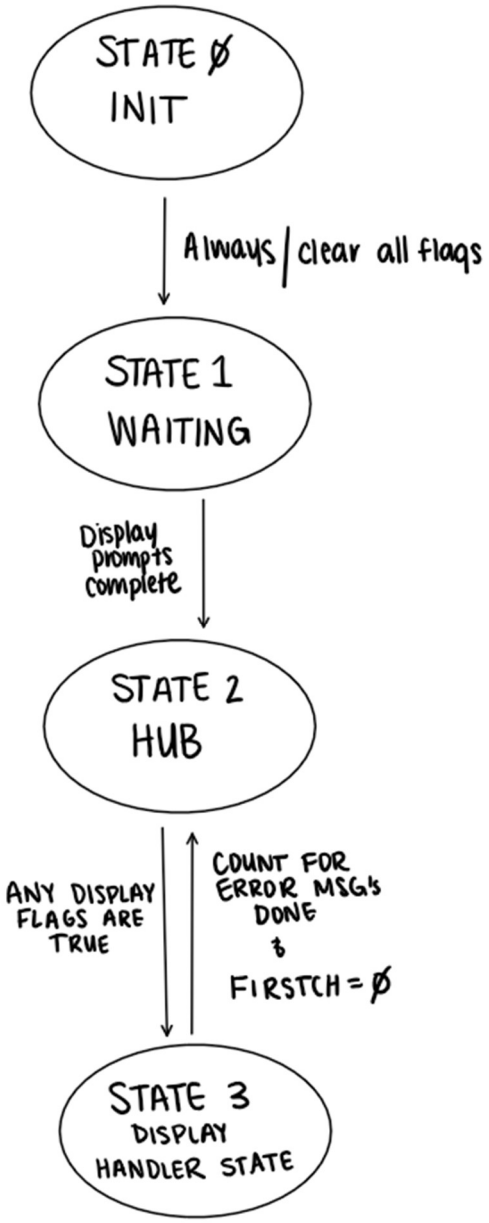
Task #1: MasterMind



Task #2: Keypad Driver



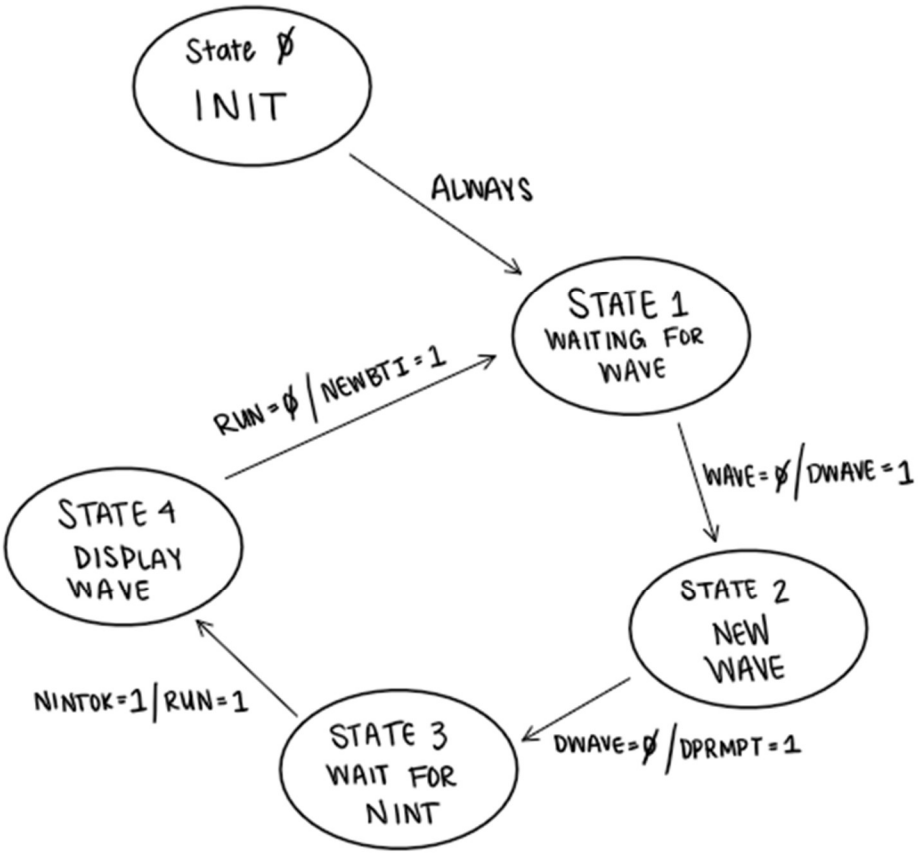
Task #3: Display



Task #4: Timer Channel 0 Set-Up

No Finite State Machine since it just runs through the one initialization and then doesn't come back to this task.

Task #5: Function Generator



Wave Form Data Tables and Oscilloscope Images:

Sawtooth:

Code:	Explanation:
DC.B 2	Number of segments for sawtooth wave
DC.W 0	Initial DAV input value, equal to 0 volts
DC.B 19	Length of segment_1
DC.W 205	Increment for segment_1
DC.B 1	Length of segment_2
DC.W 200	Increment for segment_2

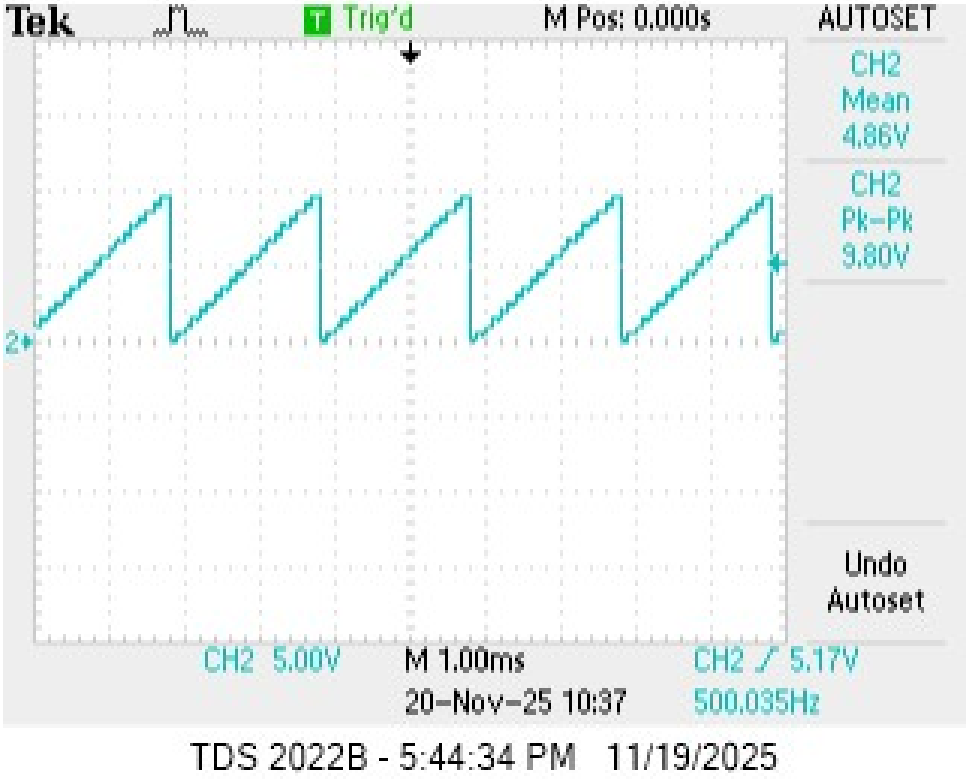


Figure 1. Oscilloscope image of our program displaying a sawtooth wave at an NINT of 1.

Square:

Code:	Explanation:
DC.B 4	Number of segments for square wave
DC.W 0	Initial DAV input value, equal to 0 volts
DC.B 1	Length for segment_1
DC.W 4095	Increment for segment_1
DC.B 9	Length for segment_2
DC.W 0	Increment for segment_2
DC.B 1	Length for segment_3
DC.W -4095	Increment for segment_3
DC.B 9	Length for segment_4
DC.W 0	Increment for segment_4

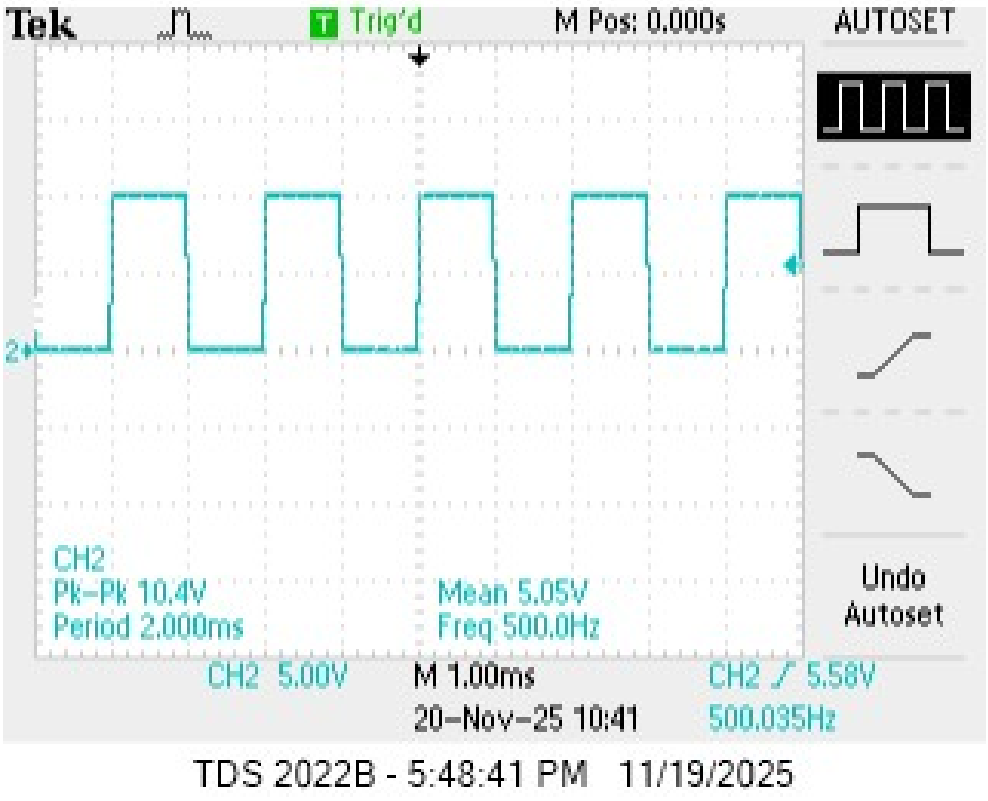


Figure 2. Oscilloscope image of our program displaying a square wave at an NINT of 1.

Sine-7:

Code:	Explanation:
DC.B 7	Number of segments for sine wave
DC.W 2048	Initial DAV input value, equal to 5 volts
DC.B 25	Length for segment_1
DC.W 33	Increment for segment_1
DC.B 50	Length for segment_2
DC.W 8	Increment for segment_2
DC.B 50	Length for segment_3
DC.W -8	Increment for segment_3
DC.B 50	Length for segment_4
DC.W -33	Increment for segment_4
DC.B 50	Length for segment_5
DC.W -8	Increment for segment_5
DC.B 50	Length for segment_6
DC.W 8	Increment for segment_6
DC.B 25	Length for segment_7
DC.W 33	Increment for segment_7

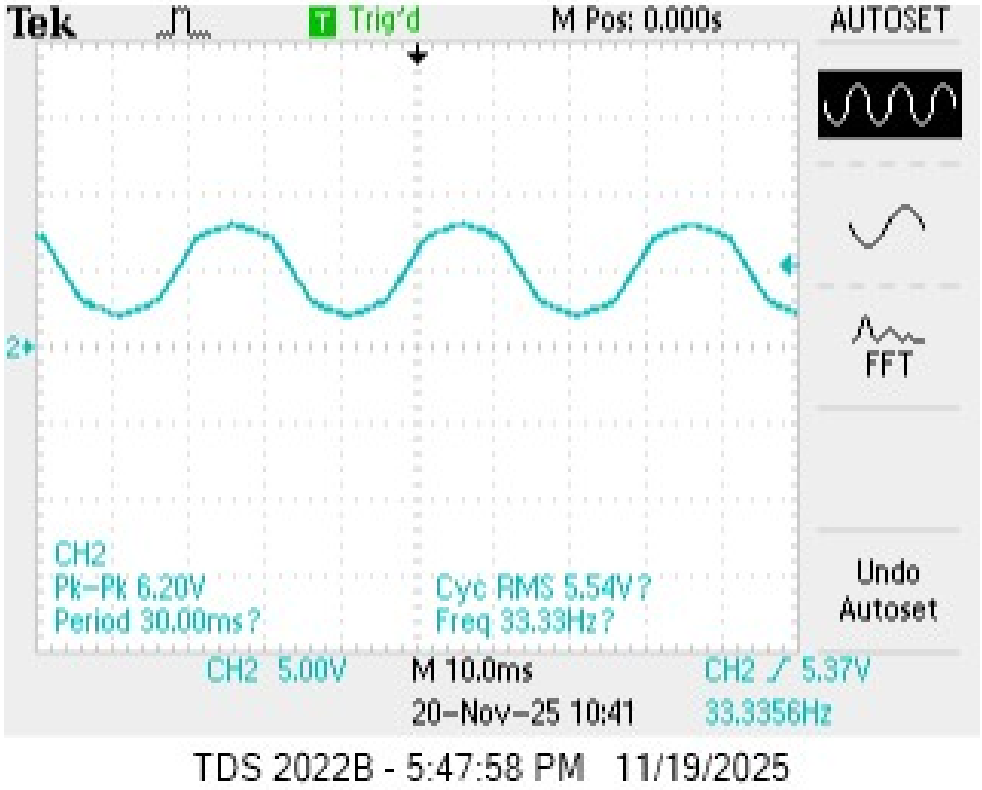


Figure 1. Oscilloscope image of our program displaying a sine wave composed of 7 segments at an NINT of 1.

Sine-15:

Code:	Explanation:
DC.B 15	Number of segments for sine wave
DC.W 2048	Initial DAV input value, equal to 5 volts
DC.B 10	Length for segment_1
DC.W 41	Increment for segment_1
DC.B 21	Length for segment_2
DC.W 37	Increment for segment_2
DC.B 21	Length for segment_3
DC.W 25	Increment for segment_3
DC.B 21	Length for segment_4
DC.W 9	Increment for segment_4
DC.B 21	Length for segment_5
DC.W -9	Increment for segment_5
DC.B 21	Length for segment_6
DC.W -25	Increment for segment_6
DC.B 21	Length for segment_7
DC.W -37	Increment for segment_7
DC.B 20	Length for segment_8
DC.W -41	Increment for segment_8
DC.B 21	Length for segment_9
DC.W -37	Increment for segment_9

DC.B 21	Length for segment_10
DC.W -25	Increment for segment_10
DC.B 21	Length for segment_11
DC.W -9	Increment for segment_11
DC.B 21	Length for segment_12
DC.W 9	Increment for segment_12
DC.B 21	Length for segment_13
DC.W 25	Increment for segment_13
DC.B 21	Length for segment_14
DC.W 37	Increment for segment_14
DC.B 10	Length for segment_15
DC.W 41	Increment for segment_15

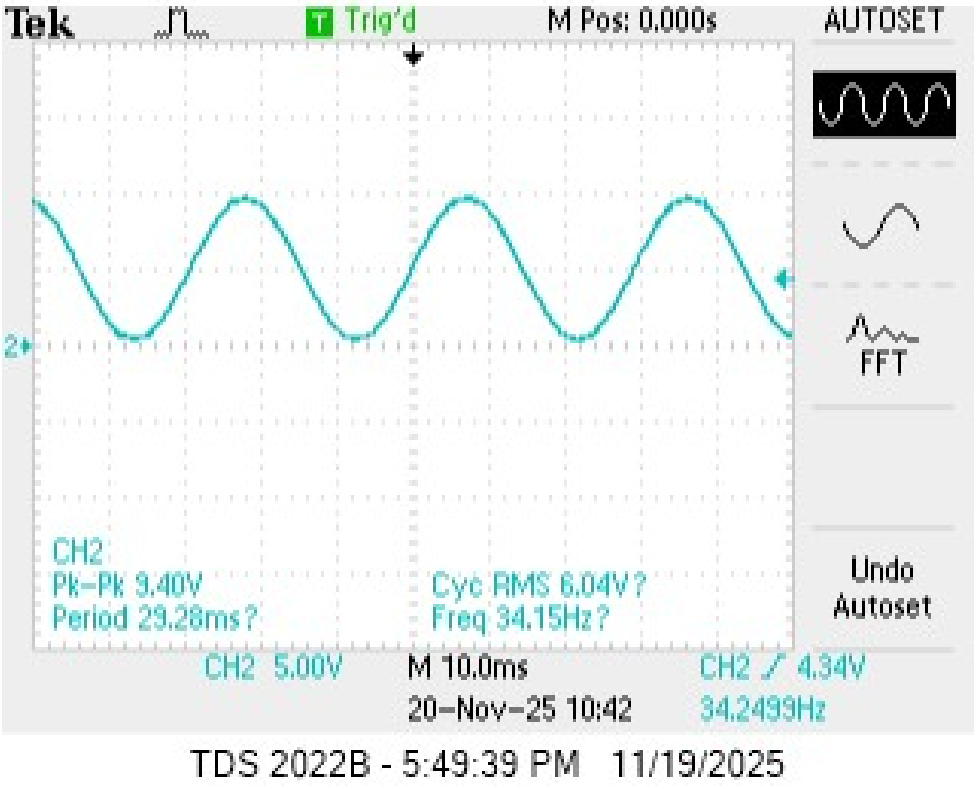


Figure 4. Oscilloscope image of our program displaying a sine wave composed of 15 segments at an NINT of 1.

```
;*******************************************************************************
;* Lab #4 Main [includes LibV4.0]                                              *
;*******************************************************************************
;* Summary:                                                                    *
;*   This program implements a multi-waveform function generator with the following   *
;*   features:                                                                 *
;*                                                                             *
;*   - Generates 4 waveform types: Sawtooth, 7-segment Sine, Square, 15-segment Sine  *
;*   - User selects waveform via keypad (keys 1-4)                             *
;*   - Prompts user to enter NINT value (1-255) controlling waveform period    *
;*   - Validates NINT input with error checking:                               *
;*     * Zero magnitude error (value = 0)                                      *
;*     * Magnitude too large error (value > 255)                               *
;*     * No digits entered error                                               *
;*   - Outputs waveform to DAC Channel A at 0.1ms Basic Time Intervals (BTI)   *
;*   - Uses cooperative multitasking with 5 independent tasks:                 *
;*     TASK_1: MasterMind - coordinates user input and waveform selection      *
;*     TASK_2: Keypad Driver - handles keypad input                            *
;*     TASK_3: Display Manager - controls LCD output                           *
;*     TASK_4: Timer Setup - initializes Timer Channel 0 for 0.1ms interrupts  *
;*     TASK_5: Waveform Generator - manages waveform output logic              *
;*   - Timer ISR (TOC1ISR) updates DAC at each BTI based on NINT value         *
;*                                                                             *
;* Author: Gabe Choi and Maddie Howard                                         *
;* Cal Poly University                                                         *
;* Fall 2025                                                                   *
;*******************************************************************************

;/-----------------------------------------------------------------------------\
;| External Definitions                                                        |
;\-----------------------------------------------------------------------------/
              XDEF  main

              XREF  ENABLE_MOTOR, STARTUP_MOTOR, UPDATE_MOTOR, DISABLE_MOTOR
              XREF  STARTUP_PWM, STARTUP_ENCODER, READ_ENCODER, DISABLE_ENCODER
              XREF  OUTDACA, OUTDACB
              XREF  INITLCD, CLRSCREEN, SETADDR, GETADDR, CURSOR_ON, DISP_OFF
              XREF  OUTCHAR, OUTCHAR_AT, OUTSTRING, OUTSTRING_AT
              XREF  INITKEY, LKEY_FLG, GETCHAR
              XREF  Entry, ISR_KEYPAD

;/-----------------------------------------------------------------------------\
;| Assembler Equates                                                           |
;\-----------------------------------------------------------------------------/
; Timer Configuration
INTERVAL    EQU 1000        ; 0.1 ms @ bus clock (per lab)
TIOS        EQU $0040       ; Timer Input/Output Select
TCNT        EQU $0044       ; Timer Counter Register
TSCR1       EQU $0046       ; Timer System Control Register 1
TCTL1       EQU $0049       ; Timer Control Register 1
TMSK1       EQU $004C       ; Timer Interrupt Mask Register 1
TFLG1       EQU $004E       ; Timer Interrupt Flag Register 1
TC0         EQU $0050       ; Timer Channel 0 Register
C0F         EQU $01         ; Channel 0 flag bit
Chan0       EQU $01         ; Channel 0 mask

; Keypad Special Characters
BSPACE      EQU $08         ; Backspace key
ENT         EQU $0A         ; Enter key

; Error message display timer (0.1 ms ticks)
```

```
ERR_ONESEC  EQU 10000       ; 1.0 s  = 10000 * 0.1 ms

;/-------------------------------------------------------------------------------\
;| RAM Variables                                                                 |
;\-------------------------------------------------------------------------------/
DEFAULT_RAM:  SECTION

; Task State Variables
t1state:    DS.B 1          ; TASK_1 (MasterMind) state
t2state:    DS.B 1          ; TASK_2 (Keypad) state
t3state:    DS.B 1          ; TASK_3 (Display) state
t4state:    DS.B 1          ; TASK_4 (Timer Setup) state
t5state:    DS.B 1          ; TASK_5 (Function Generator) state

; NINT Input Variables
NINT:       DS.B 3          ; ASCII buffer (up to 3 digits)
COUNT:      DS.B 1          ; count of digits entered
NINT_VAL:   DS.B 1          ; validated numeric NINT (1..255)
DPRMPT:     DS.B 1          ; flag for display of NINT prompt

; Waveform Engine Variables
WAVEPTR:    DS.W 1          ; pointer to start of chosen wave table
SEGPTR:     DS.W 1          ; pointer to current segment within table
CSEG:       DS.B 1          ; remaining segments in cycle
LSEG:       DS.B 1          ; remaining BTIs in current segment
SEGINC:     DS.W 1          ; per-BTI increment for current segment
VALUE:      DS.W 1          ; current DAC value

; Communication Flags
KEY_FLG:    DS.B 1          ; set by TASK_2 when key available
KEY_BUF:    DS.B 1          ; ASCII of last key
BSPACE_FLG: DS.B 1          ; backspace flag
DIGIT_FLG:  DS.B 1          ; digit entered flag
FIRSTCH:    DS.B 1          ; first character flag for display
DPTR:       DS.W 1          ; display pointer

; Display Request Flags
WAVE_FLG:   DS.B 1          ; selected wave: 1=SAW, 2=SINE7, 3=SQUARE, 4=SINE15
DWAVE:      DS.B 1          ; display initial menu line
DSAW:       DS.B 1          ; display SAW prompt
DSINE7:     DS.B 1          ; display SINE7 prompt
DSQUARE:    DS.B 1          ; display SQUARE prompt
DSINE15:    DS.B 1          ; display SINE15 prompt
DLARGEMSG:  DS.B 1          ; display "magnitude too large" error
DZEROMSG:   DS.B 1          ; display "zero magnitude" error
DNOINPUT:   DS.B 1          ; display "no digits entered" error

; Timer and Control Variables
NEWBTI:     DS.B 1          ; legacy BTI flag (unused)
RUN:        DS.B 1          ; 1 when waveform is running
CNT:        DS.B 1          ; countdown for current BIT
NINIT:      DS.B 1          ; initial value for CNT (copy of NINT_VAL)
NEWBIT:     DS.B 1          ; flag for new BIT
OUTDAC:     DS.W 1          ; DAC output value (unused)

; ASCII to Binary Conversion Temporary Variables
TEMP_RESULT: DS.W 1         ; temporary result for conversion
TEMP_INDEX:  DS.B 1         ; temporary index for conversion

; ---- NEW: Error message timeout state ----
ERR_TMR:    DS.W 1          ; down-counter in 0.1 ms ticks
```

```
ERR_ACTIVE: DS.B 1          ; 1 while an error is currently displayed

;/-------------------------------------------------------------------------------\
;| Main Program                                                                  |
;\-------------------------------------------------------------------------------/
MyCode:     SECTION

main:
    ; Initialize all task states to 0
    clr     t1state
    clr     t2state
    clr     t3state
    clr     t4state
    clr     t5state

top:
    jsr     TASK_1          ; MasterMind
    jsr     TASK_2          ; Keypad driver
    jsr     TASK_3          ; Display manager
    jsr     TASK_4          ; Timer setup (runs once)
    jsr     TASK_5          ; Waveform generator
    bra     top

;================================================================================
; TASK_1: MasterMind - Coordinates user input and waveform selection
;================================================================================
; State Machine:
;   State 0: Initialize flags and request menu display
;   State 1: Wait for menu display completion
;   State 2: Hub - wait for wave selection (keys 1-4)
;   State 3: NINT entry - accept digits, backspace, and enter
;================================================================================
TASK_1:
    ldaa    t1state
    beq     t1state0
    deca
    beq     t1state1
    deca
    beq     t1state2
    deca
    lbeq    t1state3
    rts

;--------------------------------------------------------------------------
; State 0: Initialization
;--------------------------------------------------------------------------
t1state0:
    ; Clear all display and input flags
    clr     KEY_FLG
    clr     BSPACE_FLG
    clr     DIGIT_FLG
    clr     WAVE_FLG
    clr     DSAW
    clr     DSINE7
    clr     DSQUARE
    clr     DSINE15
    clr     DLARGEMSG
    clr     DZEROMSG
    clr     DNOINPUT
    clr     ERR_ACTIVE
    clr     ERR_TMR
```

```
    ; Request initial menu display
    movb    #$01, DWAVE

;-------------------------------------------------------------------------
; State 1: Wait for initial display completion
;-------------------------------------------------------------------------
t1state1:
    tst     DWAVE               ; Wait for display to finish wave message on first line
    bne     t1s1_hold           ; If not done yet, stay in state 0
    movb    #$02, t1state       ; Advance to hub (state 2)

t1s1_hold:
    rts

;-------------------------------------------------------------------------
; State 2: Hub - Wait for wave selection (1-4)
;-------------------------------------------------------------------------
t1state2:
    tst     KEY_FLG             ; Test Key_flag to see if a key has been pressed
    beq     t1s2_exit           ; If not, exit
    clr     KEY_FLG             ; Clear key flag and move into key identification

    ; Check which wave was selected
    ldaa    KEY_BUF             ; Load key buf
    cmpa    #'1'                ; Compare to 1
    beq     t1sel_saw           ; If 1, sawtooth waveform has been selected
    cmpa    #'2'                ; Compare to 2
    beq     t1sel_sine7         ; If 2, sine-7 waveform has been selected
    cmpa    #'3'                ; Compare to 3
    beq     t1sel_square        ; If 3, square waveform has been selected
    cmpa    #'4'                ; Compare to 4
    beq     t1sel_sine15        ; If 4, sine-15 waveform has been selected
    rts                         ; Ignore other keys and return

; Wave selection handlers
t1sel_saw:
    movb    #1, WAVE_FLG        ; Raise wave flag for function generator
    movb    #1, DSAW            ; Raise display message flag for sawtooth wave for display
    clr     COUNT               ; clear count
    movb    #3, t1state         ; Go to NINT entry
    rts                         ; Return from subroutine

t1sel_sine7:
    movb    #2, WAVE_FLG        ; Raise wave flag for function generator
    movb    #1, DSINE7          ; Raise display message flag for sine-7 wave for display
    clr     COUNT               ; clear count
    movb    #3, t1state         ; Go to NINT entry
    rts                         ; Return from subroutine

t1sel_square:
    movb    #3, WAVE_FLG        ; Raise wave flag for function generator
    movb    #1, DSQUARE         ; Raise display message flag for square wave for display
    clr     COUNT               ; clear count
    movb    #3, t1state         ; Go to NINT entry
    rts                         ; Return from subroutine

t1sel_sine15:
    movb    #4, WAVE_FLG        ; Raise wave flag for function generator
    movb    #1, DSINE15         ; Raise display message flag for sine-15 wave for display
    clr     COUNT               ; clear count
```

```
    movb    #3, t1state         ; Go to NINT entry
    rts                         ; Return from subroutine

t1s2_exit:
    rts                         ; Return from subroutine

;-------------------------------------------------------------------------------
; State 3: NINT Entry - Accept digits, backspace, enter
;-------------------------------------------------------------------------------
t1state3:
    tst     KEY_FLG             ; Test key flag
    lbeq    t1s3_exit           ; If not raised, exit state
    clr     KEY_FLG             ; Clear key flag
    ldab    KEY_BUF             ; Load key buf

    cmpb    #ENT                ; Compare to ENTER key
    beq     t1s3_enter          ; If Enter - go to ENTER substate

    cmpb    #BSPACE             ; Compare to BACKSPACE key
    beq     t1s3_back           ; If backspace - go to BACKSPACE substate

    cmpb    #'0'                ; Check if key pressed is a valid digit
    lblo    t1s3_exit           ; If not exit back to hub
    cmpb    #'9'
    lbhi    t1s3_exit

    ldaa    COUNT               ; Load count
    cmpa    #3                  ; Check if buf is full of digits
    lbhs    t1s3_exit           ; If yes, exit to hub

    ldx     #NINT               ; Load NINT
    ldaa    COUNT               ; Load count
    stab    A,X                 ; Store ASCII digit at NINT[COUNT]
    inc     COUNT               ; Increase count
    movb    #1, DIGIT_FLG       ; Raise digit flag for display
    rts                         ; Return from subroutine

t1s3_back:
    tst     COUNT               ; test count
    lbeq    t1s3_exit           ; if zero, exit back to hub
    dec     COUNT               ; decrease count
    movb    #1, BSPACE_FLG      ; raise backspace flag
    rts                         ; return from subroutine

t1s3_enter:
    tst     COUNT               ; Test count
    bne     t1s3_have           ; validate NINT value, process
    movb    #1, DNOINPUT        ; Error: no digits entered
    rts                         ; return from subroutine


t1s3_have:
    jsr     ASCII_2_BIN_8       ; Convert ASCII to binary (A=result, B=error)
    tstb                        ; test what is in accumulator B
    beq     t1s3_ok             ; if no error - NINT is okay
    cmpb    #1                  ; if error is one, NINT is too large
    beq     t1s3_large          ; branch to too large
    cmpb    #2                  ; if error is two, NINT is zero
    beq     t1s3_zero           ; branch to zero
    rts                         ; return from subroutine
```

```
t1s3_ok:
    staa    NINT_VAL            ; NINT is good, store value in NINT_VAL
    clr     COUNT               ; Clear count

                                ; Set waveform pointer based on selection
    ldaa    WAVE_FLG            ; load wave flag
    cmpa    #1                  ; compare to 1
    beq     t1set_saw           ; if one, go to set saw
    cmpa    #2                  ; compare to 2
    beq     t1set_sine7         ; if two, go to set sine-7
    cmpa    #3                  ; compare to 3
    beq     t1set_square        ; if three, go to set square
    cmpa    #4                  ; compare to 4
    beq     t1set_sine15        ; if 4, go to set sine-15
    rts                         ; return from subroutine

t1set_saw:
    ldx     #SAWTOOTH           ; load sawtooth data table in x
    bra     t1set_wave_ptr      ; branch to set wave ptr
t1set_sine7:
    ldx     #SINE_7             ; load sine-7 data table in x
    bra     t1set_wave_ptr      ; branch to set wave ptr
t1set_square:
    ldx     #SQUARE             ; load square data table in x
    bra     t1set_wave_ptr      ; branch to set wave ptr
t1set_sine15:
    ldx     #SINE_15            ; load sine-15 data table in x

t1set_wave_ptr:
    stx     WAVEPTR
    movb    NINT_VAL, NINIT     ; Initialize BTI counter values
    movb    NINT_VAL, CNT
    movb    #1, RUN             ; Start waveform generation
    movb    #2, t1state         ; Return to hub
    rts                         ; return from subroutine

t1s3_large:
    movb    #1, DLARGEMSG       ; raise NINT value too large flag message for display
    clr     COUNT               ; clear count
    clr     NINT                ; clear NINT
    clr     NINT+1
    clr     NINT+2
    rts                         ; Return from subroutine

t1s3_zero:
    movb    #1, DZEROMSG        ; raise NINT value zero flag message for display
    clr     COUNT               ; clear count
    clr     NINT                ; clear NINT
    clr     NINT+1
    clr     NINT+2
    rts                         ; Return from subroutine

t1s3_exit:
    rts                         ; exit state 3

;===============================================================================
; TASK_2: Keypad Driver
;===============================================================================
; State Machine:
;   State 0: Initialize keypad
;   State 1: Wait for key press
```

```
;   State 2: Wait for key to be consumed by TASK_1
;===============================================================================
TASK_2:
    ldaa    t2state
    beq     t2state0
    deca
    beq     t2state1
    deca
    beq     t2state2
    rts

;-------------------------------------------------------------------------
; State 0: Initialize keypad
;-------------------------------------------------------------------------
t2state0:
    clr     KEY_FLG              ; clear key flag
    jsr     INITKEY              ; initialize the keypad
    movb    #1, t2state          ; move to state 1
    rts                          ; return from subroutine

;-------------------------------------------------------------------------
; State 1: Wait for key press
;-------------------------------------------------------------------------
t2state1:
    tst     LKEY_FLG            ; Check library key flag
    bne     t2s1_have           ; if there is a key, process it
    rts                         ; return from subroutine

t2s1_have:
    jsr     GETCHAR             ; Get key into B
    stab    KEY_BUF             ; store into key buf
    movb    #1, KEY_FLG         ; Signal key available
    movb    #2, t2state         ; move to state 2
    rts                         ; return from subroutine

;-------------------------------------------------------------------------
; State 2: Wait for key to be consumed
;-------------------------------------------------------------------------
t2state2:
    tst     KEY_FLG             ; test for key flag to be cleared by mastermind
    beq     t2s2_ready          ; when cleared move to ready
    rts                         ; return from subroutine

t2s2_ready:
    movb    #1, t2state         ; Key consumed, back to waiting
    rts                         ; return from subroutine



;===============================================================================
; TASK_3: Display Manager - Controls all LCD output
;===============================================================================
; State Machine:
;   State 0: Initialize LCD
;   State 1: Display hub - routes to appropriate display handler
;===============================================================================
TASK_3:
    ldaa    t3state
    beq     t3state0
    deca
    beq     t3state1
```

```
    rts

;-------------------------------------------------------------------------------
; State 0: Initialize LCD
;-------------------------------------------------------------------------------
t3state0:
    jsr     INITLCD              ; initialize LDC screen
    movb    #1, FIRSTCH          ; raise flag for FIRSTCH
    movb    #1, t3state          ; move to state 1
    rts                          ; return from subroutine

;-------------------------------------------------------------------------------
; State 1: Display Hub - Check flags and route to handlers
;-------------------------------------------------------------------------------
t3state1:
    tst     ERR_ACTIVE             ; test to see if ERROR message is active
    beq     t3_disp_route          ; if not, move on to display flags
    ldd     ERR_TMR                ; load ERROR time
    bne     t3_disp_route          ; still counting. exit to display route
    clr     ERR_ACTIVE             ; error time expired, clear error
    movb    #1, DPRMPT             ; show prompt line

t3_disp_route:
                                   ; Check all display flags to see what needs to be displayed
    tst     DWAVE                  ; 1st line of wave prompts
    bne     t3_menu
    tst     DPRMPT                 ; 2nd line of wave prompt
    bne     t3_show_prompt
    tst     DSAW                   ; saw wave
    lbne    t3_saw
    tst     DSINE7                 ; sine-7 wave
    lbne    t3_sine7
    tst     DSQUARE                ; square wave
    lbne    t3_square
    tst     DSINE15                ; sine-15 wave
    lbne    t3_sine15
    tst     DIGIT_FLG              ; digit
    lbne    t3_echo_digit
    tst     BSPACE_FLG             ; backspace
    lbne    t3_backspace
    tst     DLARGEMSG              ; too large error
    lbne    t3_err_large
    tst     DZEROMSG               ; zero magnitude error
    lbne    t3_err_zero
    tst     DNOINPUT               ; no digits entered error
    lbne    t3_err_noinput
    rts                            ; Return to subroutine

;-------------------------------------------------------------------------------
; Display Handlers
;-------------------------------------------------------------------------------

                                  ; Display initial menu (top line)
t3_menu:
    ldaa    FIRSTCH
    cmpa    #1
    bne     t3_menu_cont
    ldx     #WAVEMSG
    ldaa    #$00                  ; Top line address
    jsr     PUTCHAR_1ST
    bra     t3_menu_check
```

```
t3_menu_cont:
    jsr     PUTCHAR
t3_menu_check:
    tst     FIRSTCH
    beq     t3_menu_exit
    clr     DWAVE
    movb    #$02, t1state         ; Signal TASK_1 menu is done
t3_menu_exit:
    rts

                                  ; Display NINT prompt based on selected wave
t3_show_prompt:
    ldaa    WAVE_FLG
    cmpa    #1
    bne     t3_prmpt_2
    movb    #1, DSAW
    clr     DPRMPT
    rts
t3_prmpt_2:
    cmpa    #2
    bne     t3_prmpt_3
    movb    #1, DSINE7
    clr     DPRMPT
    rts
t3_prmpt_3:
    cmpa    #3
    bne     t3_prmpt_4
    movb    #1, DSQUARE
    clr     DPRMPT
    rts
t3_prmpt_4:
    movb    #1, DSINE15
    clr     DPRMPT
    rts

                                  ; Display SAWTOOTH wave prompt (bottom line)
t3_saw:
    ldaa    FIRSTCH
    cmpa    #1
    bne     t3_saw_cont
    ldx     #SAWMSG
    ldaa    #$40                  ; Bottom line address
    jsr     PUTCHAR_1ST
    bra     t3_saw_check
t3_saw_cont:
    jsr     PUTCHAR
t3_saw_check:
    tst     FIRSTCH
    beq     t3_saw_exit
    clr     DSAW
t3_saw_exit:
    rts

                                  ; Display SINE-7 wave prompt (bottom line)
t3_sine7:
    ldaa    FIRSTCH
    cmpa    #1
    bne     t3_sine7_cont
    ldx     #SINE7MSG
    ldaa    #$40
    jsr     PUTCHAR_1ST
```

```
    bra     t3_sine7_check
t3_sine7_cont:
    jsr     PUTCHAR
t3_sine7_check:
    tst     FIRSTCH
    beq     t3_sine7_exit
    clr     DSINE7
t3_sine7_exit:
    rts

                                    ; Display SQUARE wave prompt (bottom line)
t3_square:
    ldaa    FIRSTCH
    cmpa    #1
    bne     t3_square_cont
    ldx     #SQUAREMSG
    ldaa    #$40
    jsr     PUTCHAR_1ST
    bra     t3_square_check
t3_square_cont:
    jsr     PUTCHAR
t3_square_check:
    tst     FIRSTCH
    beq     t3_square_exit
    clr     DSQUARE
t3_square_exit:
    rts

                                     ; Display SINE-15 wave prompt (bottom line)
t3_sine15:
    ldaa    FIRSTCH
    cmpa    #1
    bne     t3_sine15_cont
    ldx     #SINE15MSG
    ldaa    #$40
    jsr     PUTCHAR_1ST
    bra     t3_sine15_check
t3_sine15_cont:
    jsr     PUTCHAR
t3_sine15_check:
    tst     FIRSTCH
    beq     t3_sine15_exit
    clr     DSINE15
t3_sine15_exit:
    rts

                                ; Echo a digit to the display
t3_echo_digit:
    ldaa    COUNT
    beq     t3_echo_done
    deca                        ; Convert to 0-based index
    ldx     #NINT
    ldab    A,X                 ; Get ASCII digit
    adda    #$53                ; Position after "NINT: "
    jsr     OUTCHAR_AT
t3_echo_done:
    clr     DIGIT_FLG
    rts

                                ; Handle backspace on display
t3_backspace:
```

```
    ldaa    COUNT               ; COUNT is already decremented
    ldab    #$53
    aba                         ; Calculate position
    ldx     #BSPMSG             ; Space character
    jsr     PUTCHAR_1ST
    clr     BSPACE_FLG
    rts

                                ; Display "MAGNITUDE TOO LARGE" error
t3_err_large:
    ldaa    FIRSTCH
    cmpa    #1
    bne     t3_err_large_cont
    ldx     #LARGEMSG
    ldaa    #$53
    jsr     PUTCHAR_1ST
    bra     t3_err_large_chk
t3_err_large_cont:
    jsr     PUTCHAR
t3_err_large_chk:
    tst     FIRSTCH
    beq     t3_err_large_exit
                                ; Timeout for error message
    ldd     #ERR_ONESEC
    std     ERR_TMR
    movb    #1, ERR_ACTIVE
    clr     DLARGEMSG
t3_err_large_exit:
    rts

                                ; Display "ZERO MAGNITUDE ERROR" error
t3_err_zero:
    ldaa    FIRSTCH
    cmpa    #1
    bne     t3_err_zero_cont
    ldx     #ZEROMSG
    ldaa    #$53
    jsr     PUTCHAR_1ST
    bra     t3_err_zero_chk
t3_err_zero_cont:
    jsr     PUTCHAR
t3_err_zero_chk:
    tst     FIRSTCH
    beq     t3_err_zero_exit
                                ; Timeout for error message
    ldd     #ERR_ONESEC
    std     ERR_TMR
    movb    #1, ERR_ACTIVE
    clr     DZEROMSG
t3_err_zero_exit:
    rts

                                ; Display "NO DIGITS ENTERED" error
t3_err_noinput:
    ldaa    FIRSTCH
    cmpa    #1
    bne     t3_err_noinput_cont
    ldx     #NOINPUT
    ldaa    #$53
    jsr     PUTCHAR_1ST
    bra     t3_err_noinput_chk
```

```
t3_err_noinput_cont:
    jsr     PUTCHAR
t3_err_noinput_chk:
    tst     FIRSTCH
    beq     t3_err_noinput_exit
                                ; Timeout for error message
    ldd     #ERR_ONESEC
    std     ERR_TMR
    movb    #1, ERR_ACTIVE
    clr     DNOINPUT
t3_err_noinput_exit:
    rts

;================================================================================
; TASK_4: Timer Channel 0 Setup (ONE-TIME INITIALIZATION)
;================================================================================
; Configures Timer Channel 0 for output compare interrupts every 0.1 ms
; Uses t4state to ensure setup code runs only once
;================================================================================
TASK_4:
    tst     t4state             ; Check if already initialized
    bne     t4_skip

                                ; One-time timer configuration
    bset    TIOS, Chan0         ; Set OC mode on channel 0
    bclr    TCTL1, $02          ; Toggle action setup
    bset    TCTL1, $01
    bset    TFLG1, Chan0        ; Clear flag
    cli                         ; Enable interrupts globally
    bset    TMSK1, Chan0        ; Enable channel 0 interrupt
    bset    TSCR1, $A0          ; Enable timer, fast flag clear
    ldd     TCNT                ; Get current timer count
    addd    #INTERVAL           ; Add interval
    std     TC0                 ; Set first compare value

    movb    #1, t4state         ; Mark as initialized

t4_skip:
    rts

;================================================================================
; TASK_5: Function Generator - Manages waveform generation logic
;================================================================================
; State Machine:
;   State 0: Initialize
;   State 1: Wait for wave selection
;   State 2: Load waveform parameters from ROM table
;   State 3: Wait for NINT validation
;   State 4: Update waveform segments each BTI
;================================================================================
TASK_5:
    ldaa    t5state
    beq     t5state0
    deca
    beq     t5state1
    deca
    beq     t5state2
    deca
    beq     t5state3
    deca
    beq     t5state4
```

```
    rts

;--------------------------------------------------------------------------
; State 0: Initialization
;--------------------------------------------------------------------------
t5state0:
    clr     RUN
    clr     NEWBIT
    movb    #1, t5state
    rts

;--------------------------------------------------------------------------
; State 1: Wait for wave selection
;--------------------------------------------------------------------------
t5state1:
    tst     WAVE_FLG
    beq     t5s1_exit
    movb    #2, t5state         ; Wave selected, load parameters
t5s1_exit:
    rts

;--------------------------------------------------------------------------
; State 2: Load waveform parameters from ROM table
;--------------------------------------------------------------------------
t5state2:
    ldx     WAVEPTR             ; Base of waveform table
    movb    0,X, CSEG           ; [0] = number of segments
    movw    1,X, VALUE          ; [1..2] = initial DAC value
    movb    3,X, LSEG           ; [3] = first segment length (in BTIs)
    movw    4,X, SEGINC         ; [4..5] = first segment increment

                                ; Point SEGPTR to the second segment
    ldx     WAVEPTR
    leax    6,X
    stx     SEGPTR

    movb    #$01, DPRMPT        ; Ask display to show NINT prompt
    movb    #$03, t5state
    rts

;--------------------------------------------------------------------------
; State 3: Wait for NINT entry and validation
;--------------------------------------------------------------------------
t5state3:
    tst     DPRMPT              ; Wait for prompt to finish
    bne     t5s3_exit

    tst     RUN                 ; Wait until NINT validated and RUN=1
    beq     t5s3_exit

                                ; Copy NINT into counters
    movb    NINT_VAL, NINIT
    movb    NINT_VAL, CNT
    movb    #4, t5state
t5s3_exit:
    rts

;--------------------------------------------------------------------------
; State 4: Waveform generation - Update segments each BTI
;   IMPORTANT ORDER:
;   1) Apply SEGINC to VALUE
```

```
;   2) Decrement LSEG; if zero, advance segment (or rewind cycle)
;-------------------------------------------------------------------------
t5state4:
    tst     RUN
    beq     t5s4_idle               ; Not running
    tst     NEWBIT
    beq     t5s4_done               ; No new BTI

                                    ; Apply this BTI's increment
    ldd     VALUE
    addd    SEGINC
    std     VALUE

                                    ; Segment countdown/advance
    dec     LSEG
    bne     t5s4_clearflag          ; Still inside this segment

                                    ; End of this segment
    dec     CSEG
    bne     t5s4_nextseg            ; More segments remain this cycle

                                    ; End of cycle: reload first segment
    ldx     WAVEPTR
    movb    0,X, CSEG               ; reset segment count
    movw    1,X, VALUE              ; reset initial DAC value
    movb    3,X, LSEG               ; first segment length
    movw    4,X, SEGINC             ; first segment increment
    ldx     WAVEPTR
    leax    6,X                     ; point to second segment
    stx     SEGPTR
    bra     t5s4_clearflag

t5s4_nextseg:
                                    ; Pull the next segment from SEGPTR
    ldx     SEGPTR
    movb    0,X, LSEG
    movw    1,X, SEGINC
    leax    3,X
    stx     SEGPTR

t5s4_clearflag:
    clr     NEWBIT                   ; done with this BTI
t5s4_done:
    rts

t5s4_idle:
    movb    #1, t5state
    rts

;/--------------------------------------------------------------------------------\
; Subroutines                                                                    |
;\--------------------------------------------------------------------------------/

;-------------------------------------------------------------
; PUTCHAR_1ST / PUTCHAR
; Character-by-character display routine
; Entry:  X = pointer to string, A = LCD address (for PUTCHAR_1ST)
; Exit:   FIRSTCH flag set when string complete
;-------------------------------------------------------------
PUTCHAR_1ST:
    stx     DPTR
```

```
    jsr     SETADDR
    clr     FIRSTCH

PUTCHAR:
    ldx     DPTR
    ldab    0,X
    beq     disp_done
    incw    DPTR
    jsr     OUTCHAR
    rts

disp_done:
    movb    #1, FIRSTCH
    rts

;===============================================================================
; TOC1ISR - Timer Channel 0 Output Compare Interrupt Service Routine
;===============================================================================
; Fires every 0.1 ms (INTERVAL clock cycles)
; Counts down CNT from NINIT, outputs VALUE to DAC when CNT reaches 0
; Sets NEWBIT flag to signal TASK_5 that a new BTI has occurred
; Also services the non-blocking error-message timer (ERR_TMR)
;===============================================================================
TOC1ISR:
    ldd     ERR_TMR
    beq     _skip_err_dec
    subd    #1
    std     ERR_TMR
_skip_err_dec:

    tst     RUN                 ; Check if waveform is running
    beq     NOT_YET

    dec     CNT                 ; Decrement BTI counter
    bne     NOT_YET

                                ; BTI complete - output to DAC
    ldd     VALUE
    jsr     OUTDACA             ; Send value to DAC Channel A
    movb    NINIT, CNT          ; Reset counter
    movb    #$01, NEWBIT        ; Signal new BTI to TASK_5

NOT_YET:
                                ; Schedule next interrupt
    ldd     TC0
    addd    #INTERVAL
    std     TC0
    bset    TFLG1, C0F          ; Clear interrupt flag
    rti

;=============================================================
; ASCII_2_BIN_8
; Converts ASCII digits in NINT buffer to 8-bit binary value
; Input:   COUNT = number of digits in NINT buffer
;          NINT = buffer containing ASCII digits
; Output:  A = binary result (0-255)
;          B = error code (0=ok, 1=too large, 2=zero/invalid)
; Uses:    TEMP_RESULT, TEMP_INDEX for intermediate calculations
;=============================================================
ASCII_2_BIN_8:
    pshx
```

```
    pshy

                                ; Check for empty input
    tst     COUNT
    beq     a2b_no_input

                                ; Initialize conversion
    clr     TEMP_INDEX
    clr     TEMP_RESULT
    clr     TEMP_RESULT+1
    ldx     #NINT

a2b_loop:
                                ; Check if done processing all digits
    ldaa    TEMP_INDEX
    cmpa    COUNT
    bhs     a2b_done

                                ; Get next digit and validate
    ldab    A,X
    subb    #'0'
    blo     a2b_bad_digit
    cmpb    #9
    bhi     a2b_bad_digit

                                ; Multiply result by 10
    pshb                        ; Save digit
    ldd     TEMP_RESULT
    ldy     #10
    emul                        ; Y:D = TEMP_RESULT * 10
    cpy     #0
    bne     a2b_overflow        ; Overflow if Y != 0
    std     TEMP_RESULT

                                ; Add digit to result
    pulb
    clra
    addd    TEMP_RESULT
    bcs     a2b_overflow_no_pull
    cpd     #255
    bhi     a2b_overflow_no_pull
    std     TEMP_RESULT

                                ; Next digit
    inc     TEMP_INDEX
    bra     a2b_loop

a2b_done:
    ldd     TEMP_RESULT
    cpd     #0
    beq     a2b_is_zero

                                ; Success
    tba                         ; A = result (low byte)
    clrb                        ; B = 0 (no error)
    bra     a2b_exit

a2b_overflow:
    pula                        ; Clean up stack
a2b_overflow_no_pull:
    clra
```

```
    ldab    #1                  ; Error code 1 = too large
    bra     a2b_exit

a2b_is_zero:
    clra
    ldab    #2                  ; Error code 2 = zero
    bra     a2b_exit

a2b_bad_digit:
    clra
    ldab    #2                  ; Error code 2 = invalid
    bra     a2b_exit

a2b_no_input:
    clra
    ldab    #2                  ; Error code 2 = no input

a2b_exit:
    puly
    pulx
    rts

;/---------------------------------------------------------------------------------\
; Display Messages                                                                 |
;\---------------------------------------------------------------------------------/
WAVEMSG:    DC.B '1: SAW, 2: SINE-7, 3: SQUARE, 4: SINE-15', $00
SAWMSG:     DC.B 'SAWTOOTH WAVE NINT:           [1-->255]', $00
SINE7MSG:   DC.B 'SINE-7 WAVE   NINT:           [1-->255]', $00
SQUAREMSG:  DC.B 'SQUARE WAVE   NINT:           [1-->255]', $00
SINE15MSG:  DC.B 'SINE-15 WAVE  NINT:           [1-->255]', $00

BSPMSG:     DC.B ' ', $00
ZEROMSG:    DC.B 'ZERO MAGNITUDE ERROR ', $00
LARGEMSG:   DC.B 'MAGNITUDE TOO LARGE ', $00
NOINPUT:    DC.B 'NO DIGITS ENTERED ', $00

;/---------------------------------------------------------------------------------\
; Waveform Data Tables (ROM)                                                       |
;\---------------------------------------------------------------------------------/
; Format:
;   Byte 0:        Number of segments in waveform
;   Bytes 1-2:     Initial DAC value (word)
;   For each segment (3 bytes):
;   Byte 0:      Segment length in BTIs
;   Bytes 1-2:   Increment per BTI (signed word)

; SAWTOOTH: Linear ramp from 0V to ~8V over 255 BTIs
SAWTOOTH:
    DC.B    2               ; 1 segment
    DC.W    0               ; Initial value = 0
    DC.B    19             ; Segment length = 255 BTIs
    DC.W    205               ; Increment = 32 per BTI
    DC.B    1             ; Segment length = 255 BTIs
    DC.W    200               ; Increment = 32 per BTI

; SQUARE: 50% duty cycle square wave
SQUARE:
    DC.B    4            ; 4 segments: jump up, hold, jump down, hold
    DC.W    0
    DC.B    1
    DC.W    4095
```

```
    DC.B    9
    DC.W    0
    DC.B    1
    DC.W    -4095
    DC.B    9
    DC.W    0

; TRIANGLE: Example 3-segment waveform
TRIANGLE:
    DC.B    3               ; 3 segments
    DC.W    2048            ; Initial value = mid-range
    DC.B    50              ; Rise segment
    DC.W    30
    DC.B    100             ; Fall segment
    DC.W    -30
    DC.B    50              ; Rise segment
    DC.W    30

; SINE_7: 7-segment sine wave approximation
SINE_7:
    DC.B    7               ; 7 segments
    DC.W    2048            ; Initial value = mid-range
    DC.B    25
    DC.W    33
    DC.B    50
    DC.W    8
    DC.B    50
    DC.W    -8
    DC.B    50
    DC.W    -33
    DC.B    50
    DC.W    -8
    DC.B    50
    DC.W    8
    DC.B    25
    DC.W    33

; SINE_15: 15-segment sine wave approximation
SINE_15:
    DC.B    15              ; 15 segments
    DC.W    2048            ; Initial value = mid-range
    DC.B    10
    DC.W    41
    DC.B    21
    DC.W    37
    DC.B    21
    DC.W    25
    DC.B    21
    DC.W    9
    DC.B    21
    DC.W    -9
    DC.B    21
    DC.W    -25
    DC.B    21
    DC.W    -37
    DC.B    20
    DC.W    -41
    DC.B    21
    DC.W    -37
    DC.B    21
    DC.W    -25
```

```
    DC.B    21
    DC.W    -9
    DC.B    21
    DC.W    9
    DC.B    21
    DC.W    25
    DC.B    21
    DC.W    37
    DC.B    10
    DC.W    41

;/---------------------------------------------------------------------------------\
; Interrupt Vectors                                                               |
;\---------------------------------------------------------------------------------/
        ORG     $FFFE           ; Reset vector
        DC.W    Entry
        ORG     $FFCE           ; Port J Key Wakeup
        DC.W    ISR_KEYPAD
        ORG     $FFEE           ; Timer Channel 0
        DC.W    TOC1ISR
```